



Resolución

Una vez que la nota sea entregada, encontrarás la resolución en este link de notion:

https://www.notion.so/R-NN-Pytorch-callbacks-lighning-72d0d7e2ab024a6c9e5f3adfd4ee574b?source=copy_link

Recuerda que existen varias formas de solucionar las actividades propuestas en los laboratorios. Toma la resolución entregada como referencia. En caso de que el link de resolución no esté público, solicita acceso en el mismo link de notion.

Neural Networks with PyTorch Lightning model

PyTorch Lightning is a lightweight wrapper for organizing PyTorch code in a structured and modular way. It provides a high-level interface that automates much of the boilerplate code typically required for training PyTorch models, allowing researchers and practitioners to focus more on experimenting with models and less on engineering tasks.

Here's why PyTorch Lightning is important:

1. **Structured Code:** PyTorch Lightning encourages writing organized, structured code by separating concerns into distinct components like data handling, model definition, training, and evaluation. This makes the codebase cleaner, more readable, and easier to maintain.
2. **Reproducibility:** By abstracting away much of the training loop and providing standard hooks for various stages of training, PyTorch Lightning helps ensure reproducibility across different experiments and environments. This is crucial for research and development, where reproducibility is a fundamental aspect.
3. **Reduced Boilerplate:** PyTorch Lightning eliminates a significant amount of boilerplate code that would otherwise be necessary for tasks like setting up training loops, handling distributed training, and logging metrics. This allows practitioners to focus more on designing experiments and less on writing infrastructure code.

4. **Flexibility:** Despite its high-level abstractions, PyTorch Lightning remains highly flexible, allowing users to customize and extend functionality as needed. It provides hooks and callbacks for incorporating custom logic at various stages of the training process.
5. **Community and Ecosystem:** PyTorch Lightning has a growing community of users and contributors, which means access to a wealth of resources, tutorials, and pre-built components. This fosters collaboration and accelerates development by leveraging shared knowledge and best practices.

In [2]: `%%capture
!pip install lightning`

```
Requirement already satisfied: lightning in /opt/anaconda3/lib/python3.13/site-packages (2.6.5)
Requirement already satisfied: PyYAML<8.0,>=5.4 in /opt/anaconda3/lib/python3.13/site-packages (from lightning) (6.0.3)
Requirement already satisfied: fsspec<2028.0,>=2022.5.0 in /opt/anaconda3/lib/python3.13/site-packages (from fsspec[http]<2028.0,>=2022.5.0->lightning) (2025.10.0)
Requirement already satisfied: lightning-utilities<2.0,>=0.10.0 in /opt/anaconda3/lib/python3.13/site-packages (from lightning) (0.15.3)
Requirement already satisfied: packaging<27.0,>=23.0 in /opt/anaconda3/lib/python3.13/site-packages (from lightning) (25.0)
Requirement already satisfied: torch<4.0,>=2.1.0 in /opt/anaconda3/lib/python3.13/site-packages (from lightning) (2.12.0)
Requirement already satisfied: torchmetrics<3.0,>=0.7.0 in /opt/anaconda3/lib/python3.13/site-packages (from lightning) (1.9.0)
Requirement already satisfied: tqdm<6.0,>=4.57.0 in /opt/anaconda3/lib/python3.13/site-packages (from lightning) (4.67.1)
Requirement already satisfied: typing-extensions<6.0,>4.5.0 in /opt/anaconda3/lib/python3.13/site-packages (from lightning) (4.15.0)
Requirement already satisfied: pytorch-lightning in /opt/anaconda3/lib/python3.13/site-packages (from lightning) (2.6.5)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /opt/anaconda3/lib/python3.13/site-packages (from fsspec[http]<2028.0,>=2022.5.0->lightning) (3.13.2)
Requirement already satisfied: filelock in /opt/anaconda3/lib/python3.13/site-packages (from torch<4.0,>=2.1.0->lightning) (3.20.0)
Requirement already satisfied: setuptools<82 in /opt/anaconda3/lib/python3.13/site-packages (from torch<4.0,>=2.1.0->lightning) (80.9.0)
Requirement already satisfied: sympy>=1.13.3 in /opt/anaconda3/lib/python3.13/site-packages (from torch<4.0,>=2.1.0->lightning) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /opt/anaconda3/lib/python3.13/site-packages (from torch<4.0,>=2.1.0->lightning) (3.5)
Requirement already satisfied: jinja2 in /opt/anaconda3/lib/python3.13/site-packages (from torch<4.0,>=2.1.0->lightning) (3.1.6)
Requirement already satisfied: numpy>1.20.0 in /opt/anaconda3/lib/python3.13/site-packages (from torchmetrics<3.0,>=0.7.0->lightning) (2.3.5)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /opt/anaconda3/lib/python3.13/site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->
```

```

fsspec[http]<2028.0,>=2022.5.0->lightning) (2.6.1)
Requirement already satisfied: aiohttp>=1.4.0 in /opt/anaconda3/lib/python3.13/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<2028.0,>=2022.5.0->lightning) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /opt/anaconda3/lib/python3.13/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<2028.0,>=2022.5.0->lightning) (25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in /opt/anaconda3/lib/python3.13/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<2028.0,>=2022.5.0->lightning) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /opt/anaconda3/lib/python3.13/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<2028.0,>=2022.5.0->lightning) (6.7.0)
Requirement already satisfied: propcache>=0.2.0 in /opt/anaconda3/lib/python3.13/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<2028.0,>=2022.5.0->lightning) (0.3.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /opt/anaconda3/lib/python3.13/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<2028.0,>=2022.5.0->lightning) (1.22.0)
Requirement already satisfied: idna>=2.0 in /opt/anaconda3/lib/python3.13/site-packages (from yarl<2.0,>=1.17.0->aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<2028.0,>=2022.5.0->lightning) (3.11)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /opt/anaconda3/lib/python3.13/site-packages (from sympy>=1.13.3->torch<4.0,>=2.1.0->lightning) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /opt/anaconda3/lib/python3.13/site-packages (from jinja2->torch<4.0,>=2.1.0->lightning) (3.0.2)

```

```

In [3]: import os, torch
import torch.nn as nn
import torch.utils.data as data
import torchvision as tv
from torch.utils.data import random_split, DataLoader, Subset
from torchmetrics import Accuracy
import lightning as L

```

```

/opt/anaconda3/lib/python3.13/site-packages/torch/cuda/__init__.py:63: FutureWarning: The pynvml package is deprecated. Please install nvidia-ml-py instead. If you did not install pynvml directly, please report this to the maintainers of the package that installed pynvml for you.
import pynvml # type: ignore[import]

```

```

In [4]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("You are using", device)

```

You are using cpu

Checking GPU information

```

In [5]: !nvidia-smi #only works for device = "gpu"!!!!!!

```

zsh:1: command not found: nvidia-smi

```
In [6]: !nvidia-smi -L
```

```
zsh:1: command not found: nvidia-smi
```

```
In [7]: num_gpus = 1 # num. gpus you want to use in this notebook

os.environ["CUDA_VISIBLE_DEVICES"] = ",".join(str(x) for x in range
print("CUDA_VISIBLE_DEVICES =", os.environ["CUDA_VISIBLE_DEVICES"])
```

```
CUDA_VISIBLE_DEVICES = 0
```

Custom PyTorch Lightning Data Module for Dataset Handling

This code defines a custom data module `MyDataModule` using PyTorch Lightning. Let's break down what each part does:

1. Initialization:

- The `__init__` method initializes the data module with parameters such as `batch_size`, `transform`, `num_classes`, and `input_shape`.
- It sets up a transformation pipeline using `torchvision.transforms.Compose`. This pipeline converts input data into tensors and then normalizes them.

2. Data Preparation:

- The `prepare_data` method is responsible for any one-time preparation needed for the data.
- In this case, it downloads the FashionMNIST dataset if not already downloaded and applies the transformation pipeline defined earlier.

3. Data Setup:

- The `setup` method prepares datasets for training, validation, testing, or prediction based on the stage specified.
- For the 'fit' stage (training), it splits the training dataset into training and validation sets using `random_split` from `torch.utils.data`.
- For the 'test' stage, it prepares the test dataset.
- For the 'predict' stage, it sets up a subset of the test dataset containing the first 5 elements, just for demonstration purposes.

4. Data Loaders:

- The `train_dataloader`, `val_dataloader`, `test_dataloader`, and `predict_dataloader` methods return PyTorch `DataLoader` instances for training, validation, testing, and

prediction, respectively.

- These `DataLoader` objects are responsible for batching, shuffling, and loading the data.

5. Instantiating the Data Module:

- An instance of `MyDataModule` is created with a specified `batch_size` of 32.
- `prepare_data` is called to ensure data is prepared before setup.
- `setup` is called without specifying a stage, which means it will prepare data for all stages (`'fit'`, `'test'`, and `'predict'`).

By utilizing this custom data module, you can easily organize your data loading pipeline, making it more modular and reusable within PyTorch Lightning training loops.

```
In [8]: class MyDataModule(L.LightningDataModule): #LDM
    def __init__(self, batch_size=64):
        super().__init__()
        self.batch_size = batch_size
        self.transform = tv.transforms.Compose([ #pipeline de compu
            tv.transforms.ToTensor()
            ,tv.transforms.Normalize((0.5,),(0.5,)) #normalize gray.
        ]) #pipeline for image processing

        self.num_classes = 10
        self.input_shape = 28*28 #784 pixels
    def prepare_data(self):
        # This method can be used for one-time data preparation
        # For example, downloading datasets or initializing data so

        tv.datasets.FashionMNIST(root="./data", train=True, download
        tv.datasets.FashionMNIST(root="./data", train=False, downlo

    def setup(self, stage=None):
        # Data setup for training, validation, and testing
        if stage == 'fit' or stage is None:

            #Load training set (RAM)
            train_dataset = tv.datasets.FashionMNIST(root="./data",

            # Training validation split
            train_set_size = int(len(train_dataset) * 0.8) #80% tra
            valid_set_size = len(train_dataset) - train_set_size #
            seed = torch.Generator().manual_seed(42) # random seed
            train_set, valid_set = random_split(train_dataset, [tra

            self.train_dataset = train_set
            self.val_dataset = valid_set

        if stage == 'test' or stage is None:
            test_dataset = tv.datasets.FashionMNIST(root="./data",
            self.test_dataset = test_dataset
```

```

    if stage == 'predict' or stage is None:
        #select the first 5 elements of a dataloader just for t
        test_dataset = tv.datasets.FashionMNIST(root="./data",
        self.predict_dataset = Subset(test_dataset, range(0, 5))

    def train_dataloader(self):
        return DataLoader(self.train_dataset, batch_size=self.batch

    def val_dataloader(self):
        return DataLoader(self.val_dataset, batch_size=self.batch_s

    def test_dataloader(self):
        return DataLoader(self.test_dataset, batch_size=self.batch_

    def predict_dataloader(self):
        return DataLoader(self.predict_dataset, batch_size=self.bat

dm = MyDataModule(batch_size=32)
# To access the x_dataloader we need to call prepare_data and setup
dm.prepare_data()
#dm.setup(stage="fit")
dm.setup()

```

```

100%|██████████| 26.4M/26.4M [00:04<00:00, 5.65MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 149kB/s]
100%|██████████| 4.42M/4.42M [00:09<00:00, 470kB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 22.1MB/s]

```

Visualizing the FMNIST dataset

```

In [9]: import matplotlib.pyplot as plt
import random

# Create a 3x3 subplot grid
fig, axes = plt.subplots(3, 3, figsize=(8, 8))

# Randomly select 9 images from the training dataset
random_indices = random.sample(range(len(dm.train_dataset)), 9)

# Loop through each subplot and plot an image with its label
for i, ax in enumerate(axes.flat):
    # Get image and label from the dataset
    img, labelidx = dm.train_dataset[random_indices[i]]
    img = img.view(28, 28, 1)

    # Plot the image
    ax.imshow(img.squeeze(), cmap='gray')

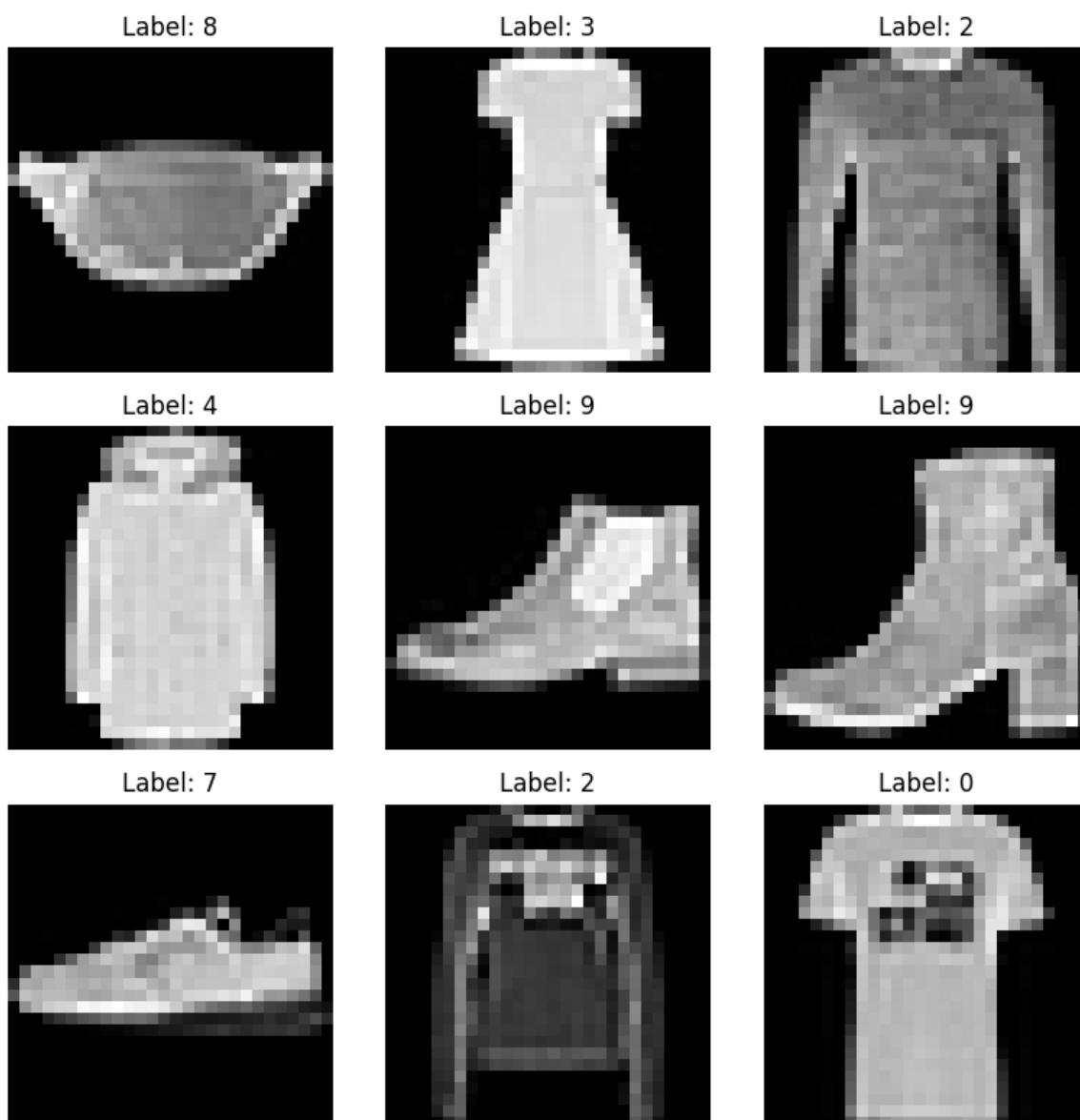
    # Print the label
    ax.set_title(f'Label: {labelidx}')

    # Remove axes
    ax.axis('off')

```

```
# Adjust layout to prevent overlap
plt.tight_layout()

# Show the plot
plt.show()
```



Let's dive deeper into the modified code to visualize a 3x3 grid of random images from the FashionMNIST dataset:

1. **Importing Libraries:** We start by importing the necessary libraries, including `matplotlib.pyplot` for visualization and `random` for generating random indices.
2. **Creating Subplots:**
 - We use `plt.subplots(3, 3, figsize=(8, 8))` to create a 3x3 grid of subplots with a specific size (8x8 inches).
 - This function returns a figure object (`fig`) and an array of axes objects (`axes`), which represent the individual subplots.
3. **Randomly Selecting Images:**

- We generate 9 random indices using `random.sample(range(len(dm.train_dataset)), 9)`.
- These indices are used to select 9 random images from the training dataset.
- `len(dm.train_dataset)` returns the total number of images in the training dataset, and `random.sample()` selects 9 unique indices from this range.

4. Plotting Images and Labels:

- We iterate over each subplot using `enumerate(axes.flat)`.
- For each subplot, we retrieve the corresponding image and label from the dataset using the randomly selected indices.
- The image is reshaped to match the dimensions expected by `imshow()` using `img.view(28, 28, 1)`.
- We plot the image using `ax.imshow()` and specify `cmap='gray'` to display it in grayscale.
- The label is printed as the title of the subplot using `ax.set_title(f'Label: {labelidx}')`.

5. Removing Axes:

- We remove the axes from each subplot using `ax.axis('off')` to clean up the visualization.

6. Adjusting Layout:

- We call `plt.tight_layout()` to adjust the layout of the subplots and prevent overlap between them.

7. Displaying the Plot:

- Finally, we use `plt.show()` to display the plot containing the 3x3 grid of images.

Understanding a PyTorch Lightning Model

This code defines a PyTorch Lightning module named `Mymodel`. Let's break down what each part does:

1. Initialization:

- The `__init__` method initializes the model with parameters such as `input_shape`, `num_classes`, and `hidden_units`.
- It defines the layers of the model using a list `layers`. The layers include a flattening layer, a series of fully connected (linear) layers with batch normalization and ReLU activation, and a final linear layer

for classification.

- It sets up the loss function (`nn.CrossEntropyLoss()`) and accuracy metric (`Accuracy`).

2. Forward Pass:

- The `forward` method defines the forward pass of the model, which applies the defined layers to the input data.

3. Optimizer Configuration:

- The `configure_optimizers` method specifies the optimizer used for training, in this case, Adam optimizer with a learning rate of 0.001.

4. Training Step:

- The `training_step` method defines the operations performed during a single training step.
- It computes the logits (raw predictions) using the forward pass, calculates the loss and accuracy, and logs them for monitoring.

5. Validation and Test Steps:

- Similar to the training step, the `validation_step` and `test_step` methods define the operations during validation and testing.
- They compute logits, calculate loss and accuracy, and log these metrics.

6. Prediction Step:

- The `predict_step` method is optional and defined here for inference purposes. It performs the forward pass without computing loss or accuracy and returns the raw predictions.

7. Instantiation:

- The model is instantiated with the specified `input_shape`, `num_classes`, and `hidden_units`.

Overall, this code provides a structured and modular way to define a neural network model for classification tasks using PyTorch Lightning, along with functionalities for training, validation, testing, and inference.

```
In [10]: class Mymodel(L.LightningModule):
def __init__(self, input_shape, num_classes, hidden_units, lambda):
    super().__init__()

    #self.classifier = nn.Sequential(
    #    nn.Flatten(),
    #    nn.Linear(input_shape, 16),
    #    nn.ReLU(),
    #    nn.Linear(16, num_classes)
    #)
```

```

self.hidden_units = hidden_units # Store the list of hidden
layers = []
layers.append(nn.Flatten()) #input layer
# Add hidden layers with ReLU activation
for units in hidden_units: #hidden layers
    layers.append(nn.Linear(input_shape, units)) #logits
    layers.append(nn.BatchNorm1d(units))
    layers.append(nn.LeakyReLU(negative_slope=0.01))
    input_shape = units
layers.append(nn.Linear(input_shape, num_classes)) #output layer
self.classifier = nn.Sequential(*layers) # here the output layer

self.lambda_L2 = lambda_L2 # L2 regularization strength
self.loss = nn.CrossEntropyLoss() #Multiclass
self.accuracy = Accuracy(task="multiclass", num_classes=num_classes)

#for inference
def forward(self, x):
    return self.classifier(x) #logits

def configure_optimizers(self):
    return torch.optim.Adam(self.parameters(), lr=0.001, weight_decay=0.0001)

def training_step(self, batch, batch_idx):
    inputs, labels = batch
    logits = self(inputs) #this calls forward. Do not use .forward
    loss = self.loss(logits, labels) # here is the softmax function

    preds = torch.argmax(logits, dim=1)
    acc = self.accuracy(preds, labels)

    self.log('train_loss', loss, on_step=True, on_epoch=True, logger=True)
    self.log('train_acc', acc, on_step=True, on_epoch=True, logger=True)
    return loss

def validation_step(self, batch, batch_idx):
    inputs, labels = batch
    logits = self(inputs) #this calls forward
    loss = self.loss(logits, labels)

    preds = torch.argmax(logits, dim=1)
    acc = self.accuracy(preds, labels)
    self.log('val_loss', loss, on_step=True, on_epoch=True, logger=True)
    self.log('val_acc', acc, on_step=True, on_epoch=True, logger=True)
    return loss

def test_step(self, batch, batch_idx):
    inputs, labels = batch
    logits = self(inputs) #this calls forward
    loss = self.loss(logits, labels)

    preds = torch.argmax(logits, dim=1)
    acc = self.accuracy(preds, labels)
    self.log('test_loss', loss, on_step=True, on_epoch=True, logger=True)
    self.log('test_acc', acc, on_step=True, on_epoch=True, logger=True)
    return loss

def predict_step(self, batch): #not necessary if dataloader with

```

```

        # this calls forward for the inputs only
        inputs, labels = batch
        return self(inputs) #return logits, this calls forward

hidden_units = [32, 64, 16] # Example hidden units list MLP, 3 hidden layers
lambda_L2 = 0.0 #regularizer
model = Mymodel(input_shape=dm.input_shape, #28*28
                 num_classes=dm.num_classes, #10
                 hidden_units=hidden_units,
                 lambda_L2=lambda_L2)#regularization L2

```

Visualizing the model

A model visualization provides valuable insights into the structure and complexity of the neural network model, helping you understand its architecture and assess its capabilities.

The first way to visualize a model is by simply printing the `model` object. In PyTorch Lightning, the `model` is typically an instance of a `LightningModule`, which encapsulates the neural network model along with training and evaluation logic. By printing the `model`, you'll see its structure and possibly any attributes it contains.

```

In [11]: print(model) #hidden_units = [32 64 16]

Mymodel(
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=784, out_features=32, bias=True)
    (2): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (3): LeakyReLU(negative_slope=0.01)
    (4): Linear(in_features=32, out_features=64, bias=True)
    (5): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (6): LeakyReLU(negative_slope=0.01)
    (7): Linear(in_features=64, out_features=16, bias=True)
    (8): BatchNorm1d(16, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (9): LeakyReLU(negative_slope=0.01)
    (10): Linear(in_features=16, out_features=10, bias=True)
  )
  (loss): CrossEntropyLoss()
  (accuracy): MulticlassAccuracy()
)

```

Another way to visualize the model is using the `torchsummary` library

```

In [12]: from torchsummary import summary
# Use torchsummary to print the summary
summary(model.to(device), input_size=(1, 28, 28)) # input_size=(channel, height, width)

```

Layer (type)	Output Shape	Param #
Flatten-1	[-1, 784]	0
Linear-2	[-1, 32]	25,120
BatchNorm1d-3	[-1, 32]	64
LeakyReLU-4	[-1, 32]	0
Linear-5	[-1, 64]	2,112
BatchNorm1d-6	[-1, 64]	128
LeakyReLU-7	[-1, 64]	0
Linear-8	[-1, 16]	1,040
BatchNorm1d-9	[-1, 16]	32
LeakyReLU-10	[-1, 16]	0
Linear-11	[-1, 10]	170
Total params: 28,666		
Trainable params: 28,666		
Non-trainable params: 0		
Input size (MB): 0.00		
Forward/backward pass size (MB): 0.01		
Params size (MB): 0.11		
Estimated Total Size (MB): 0.12		

Let's break down the code:

```
from torchsummary import summary
```

This line imports the `summary` function from the `torchsummary` library.

This function is used to print a summary of the PyTorch model's architecture, including information about the layers, output shapes, and the total number of parameters.

```
summary(model, input_size=(1, 28, 28))
```

Here, the `summary` function is called with two arguments:

- `model`: This is the PyTorch model. It represents the underlying neural network architecture.
- `input_size=(1, 28, 28)`: This specifies the input size expected by the model. The input size is defined as a tuple `(channels, height, width)`, where `1` indicates the number of channels (grayscale image), and `28` is the height and width of the input image from FMNIST dataset.

The `summary` function will then generate and print a summary of the model's architecture, which includes details such as:

- Layer names
- Output shapes of each layer
- Number of parameters (trainable and non-trainable)
- Total number of parameters in the model

Understanding Callbacks in PyTorch Lightning for Enhanced Training Control

A callback in programming is a function or a piece of code that is passed as an argument to another function or method. It allows you to customize or extend the behavior of the function or method without modifying its actual code. In the context of machine learning frameworks like PyTorch Lightning, a callback is a piece of code that is executed at specific points during the training process, such as after each epoch or after a certain metric threshold is reached.

Concretely, we use the following callbacks

```
In [13]: # Initialize Callbacks
early_stop_callback = L.pytorch.callbacks.EarlyStopping(monitor="val_loss",
                                                         mode="min",
                                                         patience=3)
checkpoint_callback = L.pytorch.callbacks.ModelCheckpoint(save_top_k=5,
                                                         save_last=True,
                                                         monitor="val_loss",
                                                         mode='min')
```

Now, let's break down the code snippet:

1. EarlyStopping Callback:

- `early_stop_callback` is initialized with an instance of the `EarlyStopping` callback.
- This callback monitors the validation loss (`monitor="val_loss"`) and tries to minimize it (`mode="min"`).
- It waits for `patience` number of epochs (in this case, 3 epochs) before considering stopping the training if the monitored quantity (validation loss) doesn't improve.

2. ModelCheckpoint Callback:

- `checkpoint_callback` is initialized with an instance of the `ModelCheckpoint` callback.
- This callback saves the model's checkpoints during training to disk.
- It saves the top 5 models based on validation loss (`save_top_k=5`).
- Additionally, it saves the last model (`save_last=True`) after training completes.
- Similar to `EarlyStopping`, it monitors the validation loss (`monitor="val_loss"`) and tries to minimize it (`mode="min"`).

In summary, the provided code initializes two callbacks: `EarlyStopping` and `ModelCheckpoint`. These callbacks provide additional functionality to

the training process, such as stopping training early if the model's performance stops improving (`EarlyStopping`) and saving model checkpoints for later use (`ModelCheckpoint`). These callbacks enhance the flexibility and functionality of the training loop, making it easier to manage and monitor the training process effectively.

Setting up PyTorch Lightning Trainer

This code snippet demonstrates how to instantiate a PyTorch Lightning Trainer object with GPU acceleration if a CUDA-enabled GPU device is available, and without GPU acceleration if not. This flexibility allows the code to seamlessly adapt to different hardware configurations, optimizing performance based on available resources.

```
In [14]: trainer = None
print('*'*80)
print("You are using", device.type)
print('*'*80)
trainer = L.Trainer(max_epochs=50,
                    callbacks=[early_stop_callback, checkpoint_callback],
                    accelerator="gpu" if device.type == "cuda" else
                    devices=1)#only works on script mode for device.
```

```
GPU available: True (mps), used: False
TPU available: False, using: 0 TPU cores
/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/trainer/setup.py:175: GPU available but not used. You can set it by doing
`Trainer(accelerator='gpu')`.
```

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

```
*****
You are using cpu
*****
```

Explanation:

1. Checking Device Availability:

- The code checks if the `device` variable is set to `"cuda"`, indicating the presence of a CUDA-enabled GPU device.
- If `device == "cuda"`, it signifies that the code will utilize GPU acceleration for training.

2. Instantiating Trainer Object:

- If a GPU device is available (`device == "cuda"`), the Trainer object is instantiated with GPU acceleration enabled.

- `max_epochs=50` specifies that the training will run for a maximum of 50 epochs.
- `callbacks=[early_stop_callback, checkpoint_callback]` specifies a list of callbacks to be used during training, including early stopping and model checkpointing.
- `accelerator="gpu"` indicates that GPU acceleration will be utilized.
- `devices=1` specifies the number of GPU devices to use. Here, it's set to 1, indicating that only one GPU will be used for training.
- If a GPU device is not available (`device != "cuda"`), the Trainer object is instantiated without GPU acceleration.
 - The parameters remain the same as above, but without specifying the `accelerator` or `devices` parameters. By default, if GPU acceleration is not specified, PyTorch Lightning will use CPU for training.

Let's train the model

This code snippet illustrates the usage of the `fit()` method to train a PyTorch Lightning model (`model`) using data provided by a LightningDataModule (`dm`). It specifies the path for saving the model checkpoints during training (`ckpt_path='last'`) to ensure that the trained model's progress is saved for future use or evaluation.

```
In [15]: trainer.fit(model, dm,
                    #checkpoint_callback.best_model_path or 'best'.
                    #use last if it is the first time you train.
                    ckpt_path='last'
                    )
```

/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/trainer/connectors/checkpoint_connector.py:190: .fit(ckpt_path="last") is set, but there is no last checkpoint available. No checkpoint will be loaded. HINT: Set `ModelCheckpoint(..., save\_last=True)`.

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	28.7 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

```

Trainable params: 28.7 K
Non-trainable params: 0
Total params: 28.7 K
Total estimated model params size (MB): 0.115
Modules in train mode: 14
Modules in eval mode: 0
Total FLOPs: 0

```

```

Output()
/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/
LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec)`
/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/
'val_dataloader' does not have many workers which may be a bot
`num_workers` argument` to `num_workers=13` in the `DataLoader
/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/
'train_dataloader' does not have many workers which may be a b
`num_workers` argument` to `num_workers=13` in the `DataLoader

```

Explanation:

- **trainer.fit() Method:**

- `trainer.fit()` is a method provided by the PyTorch Lightning Trainer object, which is responsible for training the model.
- It takes two main arguments:
 - The first argument (`model`) is the LightningModule instance that represents the neural network model to be trained.
 - The second argument (`dm`) is the LightningDataModule instance that provides the data loaders for training, validation, and testing.
- In this case, `model` is the model object to be trained, and `dm` is the data module object that contains the data loaders necessary for training.

- **Checkpoint Path (`ckpt_path`):**

- The `ckpt_path` parameter specifies the path to load or save the model checkpoints during training.
- In this code, `ckpt_path='last'` indicates that the Trainer will save the last model checkpoint after each epoch. This checkpoint will be used to resume training if interrupted or to evaluate the model after training.
- Alternatively, the commented out line
`(#checkpoint_callback.best_model_path or 'best')`
 suggests an option to specify the path of the best model checkpoint based on a provided checkpoint callback
`(checkpoint_callback)`. It assumes that the
`best_model_path` attribute of the `checkpoint_callback`
 contains the path to the best model checkpoint saved during training. If the attribute is not available or if the callback is not used, it defaults

to `'best'`.

Evaluating the model on the validation set

This code snippet demonstrates how to use the `validate()` method of the PyTorch Lightning Trainer object to evaluate a trained model (`model`) on a validation dataset. By specifying the data loader(s) for validation and the path to the model checkpoint, the Trainer ensures that the evaluation is performed using the best model obtained during training.

```
In [16]: trainer.validate(model, data loaders=dm.val_data_loader(), ckpt_path=
```

Restoring states from the checkpoint path at /Users/taisrodriguez/Desktop/ML/LAB0s/machine-learning-1139/lab5_neural_nets_pytorch_lightning/lightning_logs/version_0/checkpoints/epoch=10-step=16500.ckpt
Loaded model weights from the checkpoint at /Users/taisrodriguez/Desktop/ML/LAB0s/machine-learning-1139/lab5_neural_nets_pytorch_lightning/lightning_logs/version_0/checkpoints/epoch=10-step=16500.ckpt
Output()

Validate metric	DataLoader 0
<code>val_acc_epoch</code>	0.8853333592414856
<code>val_loss_epoch</code>	0.3195508122444153

```
Out[16]: [{'val_loss_epoch': 0.3195508122444153, 'val_acc_epoch': 0.8853333592414856}]
```

Explanation:

- **trainer.validate() Method:**

- `trainer.validate()` is a method provided by the PyTorch Lightning Trainer object, which is used for model validation.
- It takes several arguments:
 - The first argument (`model`) is the LightningModule instance representing the neural network model to be evaluated.
 - The `data loaders` argument specifies the data loader(s) to be used for validation. In this case, it's provided as `dm.val_data_loader()`, which returns the validation data loader from the given LightningDataModule (`dm`).
 - The `ckpt_path` parameter specifies the path to load the model checkpoint before validation. In this code snippet, `ckpt_path='best'` indicates that the Trainer will load the best model checkpoint saved during training for validation. This ensures that the best-performing model is used for evaluation.

Visualize the training process

The `%tensorboard --logdir lightning_logs` command launches TensorBoard and instructs it to load the log files from the `lightning_logs` directory. TensorBoard then displays visualizations of the training progress, including metrics, loss curves, histograms of model parameters, and more, allowing you to analyze and monitor the performance of your PyTorch Lightning models interactively.

Explanation:

- The `%load_ext tensorboard` command is a Jupyter Notebook magic command used to load the TensorBoard extension within the Jupyter Notebook environment.
- **`%tensorboard` Magic Command:**
 - The `%tensorboard` command is a Jupyter Notebook magic command that allows you to launch TensorBoard directly within the Jupyter Notebook environment.
 - This command is typically used when you want to visualize logs and metrics generated during the training of machine learning models.
- **`--logdir` Argument:**
 - The `--logdir` argument specifies the directory where the log files are stored. In this case, it's set to `lightning_logs`.
 - PyTorch Lightning automatically logs various metrics, such as loss, accuracy, etc., during training, validation, and testing stages. These logs are saved in the `lightning_logs` directory by default.

```
In [17]: # Load the TensorBoard notebook extension
         # %load_ext tensorboard
         # No funciona con entornos de ejecución locales ()
         %reload_ext tensorboard
```

```
In [18]: %tensorboard --logdir "/content/lightning_logs"
```

Reusing TensorBoard on port 6006 (pid 22409), started 1 day, 21:10:40 ago. (Use '!kill 22409' to kill it.)

Test the model

This code snippet demonstrates how to use the `test()` method of the PyTorch Lightning Trainer object to evaluate a trained model (`model`) on a test dataset. By specifying the data loader(s) for testing and the path to the model checkpoint, the Trainer ensures that the evaluation is performed using the best model obtained during training.

```
In [19]: # test the model
#dm.setup(stage='test')
trainer.test(model, dataloaders=dm.test_dataloader(), ckpt_path='best')
```

Restoring states from the checkpoint path at /Users/taisrodriguez/Desktop/ML/LAB0s/machine-learning-1139/lab5_neural_nets_pytorch_lightning/lightning_logs/version_0/checkpoints/epoch=10-step=16500.ckpt
Loaded model weights from the checkpoint at /Users/taisrodriguez/Desktop/ML/LAB0s/machine-learning-1139/lab5_neural_nets_pytorch_lightning/lightning_logs/version_0/checkpoints/epoch=10-step=16500.ckpt
Output()

/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/trainer/connectors/data_connector.py:434: The 'test_dataloader' does not have many workers which may be a bottleneck. Consider increasing the value of the 'num_workers' argument to 'num_workers=13' in the 'DataLoader' to improve performance.

Test metric	DataLoader 0
test_acc_epoch	0.8769999742507935
test_loss_epoch	0.3465895652770996

```
Out[19]: [{'test_loss_epoch': 0.3465895652770996, 'test_acc_epoch': 0.8769999742507935}]
```

Explanation:

- **`trainer.test()` Method:**
 - `trainer.test()` is a method provided by the PyTorch Lightning Trainer object, used for model evaluation on a test dataset.
 - It takes several arguments:
 - The first argument (`model`) is the LightningModule instance representing the neural network model to be evaluated.
 - The `dataloaders` argument specifies the data loader(s) to be used for testing. In this case, it's provided as `dm.test_dataloader()`, which returns the test data loader from the given LightningDataModule (`dm`).

- The `ckpt_path` parameter specifies the path to load the model checkpoint before testing. In this code snippet, `ckpt_path='best'` indicates that the Trainer will load the best model checkpoint saved during training for testing. This ensures that the best-performing model is used for evaluation.

Making predictions: Inference stage

Inference, in the context of deep learning, refers to the process of using a trained neural network model to make predictions or draw conclusions from new, unseen data. During inference, input data is fed into the trained model, which processes the data through its layers to produce output predictions. These predictions could include class labels, regression values, or any other desired outcome depending on the task at hand.

In PyTorch Lightning, there are two distinct stages in which a trained model is evaluated: the **predict (inference)** stage and the **test** stage. Here's a clarification of the difference between these two stages:

1. Predict (Inference) Stage:

- **Purpose:** The predict stage is primarily focused on making predictions or inferences on new, unseen data.
- **Usage:** It is used when you want to apply the trained model to new data to obtain predictions or outputs.
- **Input:** Typically, input data is provided to the model for inference, and the model's output predictions are obtained.
- **Scenario:** This stage is commonly used in real-world applications where the trained model is deployed to make predictions on user input, images, text, or any other data.
- **Flexibility:** PyTorch Lightning provides flexibility in conducting inference, allowing you to override the `predict_step` method in your LightningModule to customize the inference process.

2. Test Stage:

- **Purpose:** The test stage is focused on evaluating the performance of the trained model on a separate test dataset.
- **Usage:** It is used to assess the generalization ability and overall performance of the model on unseen data.
- **Input:** Input data from the test dataset is provided to the model, and predictions are compared against ground-truth labels to compute evaluation metrics such as accuracy, precision, recall, etc.
- **Scenario:** This stage is commonly used during model development and experimentation to validate the model's performance before

deployment.

- **Evaluation:** PyTorch Lightning provides convenience methods like `trainer.test()` to automate the evaluation process during the test stage. These methods handle the forward pass, compute metrics, and provide evaluation results.

In PyTorch Lightning, there are multiple approaches to perform inference, each with its own advantages and considerations. These approaches range from leveraging PyTorch Lightning's built-in capabilities, such as overriding the `predict_step` method, to more manual methods like directly calling the model's `forward` method. Each approach offers flexibility and trade-offs in terms of simplicity, efficiency, and customization.

Here, we present 3 options to perform predictions. **Option 1** is the preferred method as it leverages PyTorch Lightning's built-in capabilities and is more streamlined. **Option 1** and **Option 2** are commented.

Option 1:

This option demonstrates the preferred method for making inferences in PyTorch Lightning by overriding the `predict_step` method. Here's how it works:

- **Predict Step Override:**
 - The `predict_step` method is overridden within the `LightningModule` (see above the neural network model).
 - In this method, you call the `forward` method directly. This is the same method used during training and validation but tailored specifically for inference.
 - You don't need to call `eval()` or use `torch.no_grad()` explicitly, as PyTorch Lightning handles this internally.
- **Inference Process:**
 - The `LightningDataModule` is set up for inference using `dm.setup(stage='predict')`.
 - The Trainer's `predict` method is called, passing the model and the dataloader for inference.
 - Predicted logits are obtained, and softmax is applied to obtain probabilities.
 - Predicted labels are inferred from probabilities.
 - Ground-truth labels are extracted for comparison.
 - Finally, the probabilities are visualized.

Option 2 (commented in the code below):

This option demonstrates an alternative approach where you define a custom dataloader that only returns inputs for inference:

- **Custom Dataloader:**
 - You define a custom collate function (`custom_collate`) to extract only the inputs from the batch.
 - The dataloader is set up with this custom collate function.
- **Inference Process:**
 - The Trainer's `predict` method is called, passing only the dataloader (as it now only returns inputs).
 - Predicted logits are obtained.
- **No Predict Step:**
 - Options 2 does not require a `predict_step` method like option 1. Instead, it relies on custom dataloaders.

Option 3:

This option demonstrates directly calling the `forward` method for inference through `model(inputs)` :

- **Using `forward` :**
 - You directly call the `forward` method of the model.
 - You must manually handle model evaluation (`eval()`) and gradient tracking suppression (`torch.no_grad()`).
- **Inference Process:**
 - The model is set to evaluation mode using `model.eval()` .
 - Inference is performed directly by calling the `forward` method on inputs.
 - Gradient tracking is disabled within the `torch.no_grad()` context.
 - Predicted logits are obtained.
- **No Predict Step:**
 - Options 3 does not require a `predict_step` method like option 1. Instead, it relies on direct `forward` calls for inference through `model(inputs)` . Note that by calling `model(inputs)`, you're implicitly invoking the forward method of the model.

```
In [20]: #OPTION 1 (PREFERRED). overrides predict_step that calls forward
#this calls the overridden predict_step which in turn calls forward
# no need to call eval() or no_grad()

#dm.setup(stage='predict')
```

```

pred_logits = trainer.predict(model, dataloaders=dm.predict_data_loader)
print("pred_logits[0].shape is", pred_logits[0].shape)

# Assuming 'pred_logits' is a list of tensors, e.g., [tensor1, tensor2]
# Get the predicted probs for each example in the list using softmax
predicted_probs_list = [torch.softmax(logits, dim=1) for logits in pred_logits]
print("predicted_probs_list is", predicted_probs_list)
# 'predicted_labels_list' now contains the inferred class labels for each example
predicted_labels_list = [torch.argmax(probs, dim=1) for probs in predicted_probs_list]
print("predicted_labels_list is", predicted_labels_list)
# Assuming your DataLoader returns (inputs, labels) for each batch
# Use list comprehension to extract labels for all examples in the dataset
labels_list = [labels for _, labels in dm.predict_data_loader.iter_instances()]
labels_list = labels_list[0].tolist() # because the previous step returned a list of lists
# 'labels_list' now contains the labels for all examples in the Dataset

print("labels_list is", labels_list)

#visualize the probabilities
# Stack the list of tensors into a 2D tensor
stacked_tensor = torch.stack(predicted_probs_list, dim=0).view(5,10)
# Reshape the 2D tensor into a 2D numpy array
numpy_array = stacked_tensor.numpy()
# Visualize the 2D numpy array as an image
plt.imshow(numpy_array, cmap='viridis') # You can choose a different colormap
# Convert the tensors in labels_list to strings
string_labels = [str(label) for label in labels_list]
prefixed_strings = ["Ground-truth " + string for string in string_labels]
# Set the y tick labels to the string values
plt.yticks(range(len(string_labels)), prefixed_strings)
plt.colorbar()
plt.show()

#OPTION 2 no predict_step needed. Need to define a dataloader that
def custom_collate(batch):
    # Extract only the inputs and return them as a list
    list_inputs = [item[0] for item in batch]
    stacked_inputs = torch.stack(list_inputs, dim=0)
    return stacked_inputs

imgx, labelx = test_dataset[0]
predict_dataset = data.TensorDataset(imgx)
predict_dataloader = data.DataLoader(predict_dataset, batch_size=1)
print(predict_dataloader.dataset[0])
pred_logits = trainer.predict(dataloaders=predict_dataloader)
print((pred_logits[0]))

# OPTION 3, calling directly forward.
#When using forward, you are responsible to call eval() and use the model.eval()
with torch.no_grad():
    inputs, lab = test_dataloader.dataset[0]
    pred = model(inputs) #this calls forward
    print(pred)

```

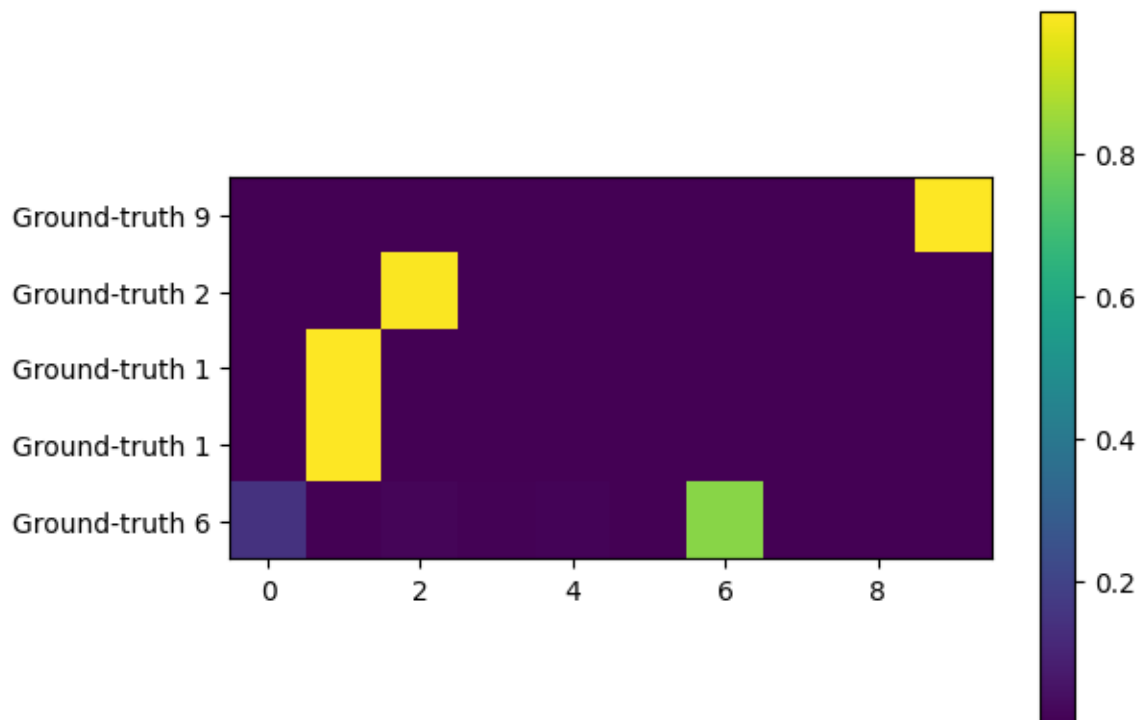

Restoring states from the checkpoint path at /Users/taisrodriguez/Desktop/ML/LAB0s/machine-learning-1139/lab5_neural_nets_pytorch_lightning/lightning_logs/version_0/checkpoints/epoch=10-step=16500.ckpt

Loaded model weights from the checkpoint at /Users/taisrodriguez/Desktop/ML/LAB0s/machine-learning-1139/lab5_neural_nets_pytorch_lightning/lightning_logs/version_0/checkpoints/epoch=10-step=16500.ckpt

Output()

/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/trainer/connectors/data_connector.py:434: The 'predict_dataloader' does not have many workers which may be a bottleneck. Consider increasing the value of the 'num_workers' argument to 'num_workers=13' in the 'DataLoader' to improve performance.

```
pred_logits[0].shape is torch.Size([5, 10])
predicted_probs_list is [tensor([[4.9066e-07, 2.7426e-06, 3.1908e-07, 1.0563e-05, 2.4892e-07, 3.6992e-04,
                                2.3499e-06, 1.3047e-03, 9.2893e-07, 9.9831e-01],
                                [6.7268e-04, 9.1339e-09, 9.9562e-01, 1.5543e-06, 2.9033e-03,
                                3.2726e-06,
                                7.9062e-04, 2.2173e-06, 5.8019e-06, 2.4546e-08],
                                [5.5965e-05, 9.9984e-01, 6.3066e-06, 4.5420e-05, 3.2026e-05,
                                2.6151e-08,
                                2.3730e-05, 6.1338e-08, 1.0005e-06, 7.8624e-09],
                                [1.0098e-05, 9.9992e-01, 5.3203e-06, 1.7213e-05, 1.0218e-05,
                                1.1509e-06,
                                3.6695e-05, 6.7451e-09, 4.1073e-07, 9.7948e-08],
                                [1.4836e-01, 2.2341e-05, 1.2771e-02, 4.8876e-03, 9.3055e-03,
                                4.5433e-04,
                                8.2373e-01, 1.9856e-06, 4.7334e-04, 2.1101e-06]])])
predicted_labels_list is [tensor([9, 2, 1, 1, 6])]
labels_list is [9, 2, 1, 1, 6]
```



Visualizing the model with Tensorboard

```
In [21]: from torch.utils.tensorboard import SummaryWriter
X_probe = torch.randn(5,28,28) #just a (batch,width, height) probe
writer = SummaryWriter("torchlogs/")
writer.add_graph(model, X_probe)
writer.close()
```

```
In [22]: %tensorboard --logdir torchlogs # requires to previously execute in
Reusing TensorBoard on port 6007 (pid 23106), started 1 day, 20:58:4
1 ago. (Use '!kill 23106' to kill it.)
```

```
In [23]: #ADVANCED VISUALIZATION WITH TORCHVIZ  
#!pip install torchviz
```

```
In [24]: #ADVANCED VISUALIZATION WITH TORCHVIZ  
#from torchviz import make_dot  
#X_probe = torch.randn(5,28,28) #just a (batch,width, height) probe  
#y = model(X_probe)
```

```
#make_dot(y.mean(), params=dict(model.named_parameters()), show_att
```

✓ Activity: K-Fold Cross-Validation

Modify the code to incorporate K-Fold Cross-Validation (CV) to optimize the number of neurons in each hidden layer. Keep the number of hidden layers fixed at 3, the L2 regularization parameter fixed at $1e-4$, and vary the number of neurons in each hidden layer using the following values: 16, 64, and 256. For this activity, utilize `sklearn.model_selection.StratifiedKFold` with a random seed of 42. Test your code with `k=5` and complete the table below with the accuracy for each run. Additionally, calculate the p-value using the model with 16 neurons as the pivot. Analyze which of the three models is the best.

	16 neurons per Hidden Layer	64 neurons per Hidden Layer	256 neurons per Hidden Layer
Fold 1			
Fold 2			
Fold 3			
Fold 4			
Fold 5			
Mean (std) -->			
p-value -->	-----		

```
In [25]: import numpy as np
import pandas as pd

from IPython.display import display
from scipy.stats import ttest_rel
from sklearn.model_selection import StratifiedKFold

# Reproducibility
L.seed_everything(42, workers=True)

# Activity settings
K = 5
RANDOM_SEED = 42
LAMBDA_L2 = 1e-4
NEURON_OPTIONS = [16, 64, 256]
MAX_EPOCHS = 10
BATCH_SIZE = 128

class KFoldDataModule(L.LightningDataModule):
    def __init__(self, dataset, train_indices, val_indices, batch_size):
        super().__init__()
        self.dataset = dataset
        self.train_indices = train_indices
        self.val_indices = val_indices
        self.batch_size = batch_size

    def setup(self, stage=None):
```

```
self.train_dataset = Subset(self.dataset, self.train_indices)
self.val_dataset = Subset(self.dataset, self.val_indices)

def train_dataloader(self):
    return DataLoader(
        self.train_dataset,
        batch_size=self.batch_size,
        shuffle=True
    )

def val_dataloader(self):
    return DataLoader(
        self.val_dataset,
        batch_size=self.batch_size,
        shuffle=False
    )

def build_trainer():
    early_stop = L.pytorch.callbacks.EarlyStopping(
        monitor="val_loss",
        mode="min",
        patience=2
    )

    return L.Trainer(
        max_epochs=MAX_EPOCHS,
        callbacks=[early_stop],
        accelerator="gpu" if torch.cuda.is_available() else "cpu",
        devices=1,
        logger=False,
        enable_checkpointing=False,
        enable_progress_bar=False,
        deterministic=True
    )

# Load the full FashionMNIST training dataset.
# K-Fold will create the train/validation splits.
dm.prepare_data()

full_dataset = tv.datasets.FashionMNIST(
    root="./data",
    train=True,
    transform=dm.transform
)

targets = np.array(full_dataset.targets)
indices = np.arange(len(full_dataset))

skf = StratifiedKFold(
    n_splits=K,
    shuffle=True,
    random_state=RANDOM_SEED
)
```

```

results = []

for fold, (train_idx, val_idx) in enumerate(skf.split(indices, target)):
    print(f"\n===== Fold {fold}/{K} =====")

    for neurons in NEURON_OPTIONS:
        print(f"Training architecture: [{neurons}, {neurons}, {neurons}]")

        L.seed_everything(RANDOM_SEED + fold, workers=True)

        fold_dm = KFoldDataModule(
            dataset=full_dataset,
            train_indices=train_idx,
            val_indices=val_idx,
            batch_size=BATCH_SIZE
        )

        fold_model = Mymodel(
            input_shape=dm.input_shape,
            num_classes=dm.num_classes,
            hidden_units=[neurons, neurons, neurons],
            lambda_L2=LAMBDA_L2
        )

        fold_trainer = build_trainer()
        fold_trainer.fit(fold_model, datamodule=fold_dm)

        metrics = fold_trainer.validate(
            fold_model,
            datamodule=fold_dm,
            verbose=False
        )[0]

        val_acc = metrics.get("val_acc_epoch", metrics.get("val_accuracy"))

        results.append({
            "fold": fold,
            "hidden_units": neurons,
            "architecture": f"[{neurons}, {neurons}, {neurons}]",
            "val_accuracy": float(val_acc)
        })

# Results table: accuracy for each fold and architecture
results_df = pd.DataFrame(results)

accuracy_table = results_df.pivot(
    index="fold",
    columns="hidden_units",
    values="val_accuracy"
)

accuracy_table.columns = [
    f"[{col}, {col}, {col}]"
    for col in accuracy_table.columns
]

```

```

print("\nAccuracy for each fold:")
display(accuracy_table.style.format("{:.4f}"))

# Summary table: mean, std, and p-values against 16 neurons
summary_table = pd.DataFrame({
    "architecture": accuracy_table.columns,
    "mean_accuracy": accuracy_table.mean(axis=0).values,
    "std_accuracy": accuracy_table.std(axis=0).values
})

pivot_column = "[16, 16, 16]"
p_values = []

for column in accuracy_table.columns:
    if column == pivot_column:
        p_values.append(np.nan)
    else:
        p_value = ttest_rel(
            accuracy_table[column],
            accuracy_table[pivot_column]
        ).pvalue
        p_values.append(p_value)

summary_table["p_value_vs_16"] = p_values

print("\nSummary:")
display(
    summary_table.style.format({
        "mean_accuracy": "{:.4f}",
        "std_accuracy": "{:.4f}",
        "p_value_vs_16": "{:.6f}"
    })
)

# Best model selection
best_row = summary_table.loc[summary_table["mean_accuracy"].idxmax()]
best_architecture = best_row["architecture"]
best_accuracy = best_row["mean_accuracy"]

print("\nBest model:")
print(f"Architecture: {best_architecture}")
print(f"Mean validation accuracy: {best_accuracy:.4f}")

# Short statistical interpretation
print("\nStatistical interpretation using [16, 16, 16] as pivot:")

for _, row in summary_table.iterrows():
    architecture = row["architecture"]
    p_value = row["p_value_vs_16"]

    if architecture == pivot_column:
        print(f"{architecture}: pivot model.")

```

```

elif p_value < 0.05:
    print(f"{architecture}: statistically significant difference")
else:
    print(f"{architecture}: no statistically significant difference")

```

Seed set to 42

```

/var/folders/l8/6j4kg7t10ldc8bv6w56t7z7w0000gn/T/ipykernel_84532/396
8698742.py:76: DeprecationWarning: __array__ implementation doesn't
accept a copy keyword, so passing copy=False failed. __array__ must
implement 'dtype' and 'copy' keyword arguments. To learn more, see t
he migration guide https://numpy.org/devdocs/numpy_2_0_migration_gui
de.html#adapting-to-changes-in-the-copy-keyword

```

```

    targets = np.array(full_dataset.targets)

```

Seed set to 43

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

```

/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/trainere
r/setup.py:175: GPU available but not used. You can set it by doing
`Trainer(accelerator='gpu')`.

```

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

===== Fold 1/5 =====

Training architecture: [16, 16, 16]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	13.4 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 13.4 K

Non-trainable params: 0

Total params: 13.4 K

Total estimated model params size (MB): 0.053

Modules in train mode: 14

Modules in eval mode: 0

Total FLOPs: 0


```

/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/utilities/_pytree.py:21: `isinstance(treespec, LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec)` and `treespec.is_leaf()` instead.
/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/trainer/connectors/data_connector.py:434: The 'val_dataloader' does not have many workers which may be a bottleneck. Consider increasing the value of the `num_workers` argument` to `num_workers=13` in the `DataLoader` to improve performance.
/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/core/module.py:522: You called `self.log('val_loss', ..., logger=True)` but have no logger configured. You can enable one by doing `Trainer(logger=ALogger(...))`
/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/core/module.py:522: You called `self.log('val_acc', ..., logger=True)` but have no logger configured. You can enable one by doing `Trainer(logger=ALogger(...))`
/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/trainer/connectors/data_connector.py:434: The 'train_dataloader' does not have many workers which may be a bottleneck. Consider increasing the value of the `num_workers` argument` to `num_workers=13` in the `DataLoader` to improve performance.
/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/core/module.py:522: You called `self.log('train_loss', ..., logger=True)` but have no logger configured. You can enable one by doing `Trainer(logger=ALogger(...))`
/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/core/module.py:522: You called `self.log('train_acc', ..., logger=True)` but have no logger configured. You can enable one by doing `Trainer(logger=ALogger(...))`
`Trainer.fit` stopped: `max_epochs=10` reached.
Seed set to 43
GPU available: True (mps), used: False
TPU available: False, using: 0 TPU cores
💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

```

Training architecture: [64, 64, 64]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	59.6 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 59.6 K

Non-trainable params: 0

Total params: 59.6 K

Total estimated model params size (MB): 0.238

Modules in train mode: 14

Modules in eval mode: 0

Total FLOPs: 0

Seed set to 43

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

Training architecture: [256, 256, 256]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	336 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 336 K

Non-trainable params: 0

Total params: 336 K

Total estimated model params size (MB): 1.347

Modules in train mode: 14

Modules in eval mode: 0

Total FLOPs: 0

Seed set to 44

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

===== Fold 2/5 =====

Training architecture: [16, 16, 16]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	13.4 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 13.4 K

Non-trainable params: 0

Total params: 13.4 K

Total estimated model params size (MB): 0.053

Modules in train mode: 14

Modules in eval mode: 0

Total FLOPs: 0

Seed set to 44

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

Training architecture: [64, 64, 64]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	59.6 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 59.6 K

Non-trainable params: 0

Total params: 59.6 K

Total estimated model params size (MB): 0.238

Modules in train mode: 14

Modules in eval mode: 0

Total FLOPs: 0

Seed set to 44

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

Training architecture: [256, 256, 256]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	336 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 336 K

Non-trainable params: 0

Total params: 336 K

Total estimated model params size (MB): 1.347

Modules in train mode: 14

Modules in eval mode: 0

Total FLOPs: 0

Seed set to 45

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

===== Fold 3/5 =====

Training architecture: [16, 16, 16]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	13.4 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 13.4 K
Non-trainable params: 0
Total params: 13.4 K
Total estimated model params size (MB): 0.053
Modules in train mode: 14
Modules in eval mode: 0
Total FLOPs: 0

Seed set to 45

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

Training architecture: [64, 64, 64]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	59.6 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 59.6 K
Non-trainable params: 0
Total params: 59.6 K
Total estimated model params size (MB): 0.238
Modules in train mode: 14
Modules in eval mode: 0
Total FLOPs: 0

Seed set to 45

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

Training architecture: [256, 256, 256]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	336 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 336 K
Non-trainable params: 0
Total params: 336 K
Total estimated model params size (MB): 1.347
Modules in train mode: 14
Modules in eval mode: 0
Total FLOPs: 0

Seed set to 46

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

===== Fold 4/5 =====

Training architecture: [16, 16, 16]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	13.4 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 13.4 K

Non-trainable params: 0

Total params: 13.4 K

Total estimated model params size (MB): 0.053

Modules in train mode: 14

Modules in eval mode: 0

Total FLOPs: 0

Seed set to 46

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

Training architecture: [64, 64, 64]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	59.6 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 59.6 K

Non-trainable params: 0

Total params: 59.6 K

Total estimated model params size (MB): 0.238

Modules in train mode: 14

Modules in eval mode: 0

Total FLOPs: 0

Seed set to 46

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

Training architecture: [256, 256, 256]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	336 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 336 K

Non-trainable params: 0

Total params: 336 K

Total estimated model params size (MB): 1.347

Modules in train mode: 14

Modules in eval mode: 0

Total FLOPs: 0

Seed set to 47

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

===== Fold 5/5 =====

Training architecture: [16, 16, 16]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	13.4 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 13.4 K

Non-trainable params: 0

Total params: 13.4 K

Total estimated model params size (MB): 0.053

Modules in train mode: 14

Modules in eval mode: 0

Total FLOPs: 0

Seed set to 47

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

Training architecture: [64, 64, 64]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	59.6 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 59.6 K
Non-trainable params: 0
Total params: 59.6 K
Total estimated model params size (MB): 0.238
Modules in train mode: 14
Modules in eval mode: 0
Total FLOPs: 0

`Trainer.fit` stopped: `max\_epochs=10` reached.

Seed set to 47

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

Training architecture: [256, 256, 256]

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	336 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 336 K
Non-trainable params: 0
Total params: 336 K
Total estimated model params size (MB): 1.347
Modules in train mode: 14
Modules in eval mode: 0
Total FLOPs: 0

Accuracy for each fold:

[16, 16, 16] [64, 64, 64] [256, 256, 256]

fold			
1	0.8735	0.8777	0.8857
2	0.8597	0.8773	0.8766
3	0.8580	0.8712	0.8823
4	0.8654	0.8870	0.8807
5	0.8695	0.8900	0.8759

Summary:

	architecture	mean_accuracy	std_accuracy	p_value_vs_16
0	[16, 16, 16]	0.8652	0.0065	nan
1	[64, 64, 64]	0.8806	0.0077	0.008239
2	[256, 256, 256]	0.8802	0.0041	0.006927

Best model:
 Architecture: [64, 64, 64]
 Mean validation accuracy: 0.8806

Statistical interpretation using [16, 16, 16] as pivot:
 [16, 16, 16]: pivot model.
 [64, 64, 64]: statistically significant difference vs [16, 16, 16] ($p=0.008239$).
 [256, 256, 256]: statistically significant difference vs [16, 16, 16] ($p=0.006927$).

Se aplicó validación cruzada estratificada con $k=5$ para comparar tres arquitecturas de una red neuronal con tres capas ocultas: [16, 16, 16], [64, 64, 64] y [256, 256, 256]. En todos los casos se mantuvo fija la regularización L2 con $\text{lambda_L2} = 1e-4$.

Los resultados muestran que la arquitectura [16, 16, 16] obtuvo una accuracy promedio de 0.8652, mientras que [64, 64, 64] alcanzó 0.8806 y [256, 256, 256] alcanzó 0.8802. Por lo tanto, las arquitecturas con mayor número de neuronas lograron un mejor desempeño promedio que el modelo base de 16 neuronas.

Al usar [16, 16, 16] como modelo pivote, se obtuvieron p-values de 0.008239 para [64, 64, 64] y 0.006927 para [256, 256, 256]. Como ambos valores son menores que 0.05, se concluye que la mejora frente al modelo de 16 neuronas es estadísticamente significativa.

El mejor modelo según la accuracy promedio fue [64, 64, 64], con 0.8806. Aunque [256, 256, 256] también tuvo un desempeño muy similar (0.8802), el modelo de 64 neuronas por capa es preferible porque obtiene la mayor accuracy promedio con una arquitectura menos compleja que la de 256 neuronas.

Activity: Implementing L1 Regularization

Modify the model to accept a new hyperparameter `Lambda_L1` to implement L1 regularization. Note that `torch.optim.Adam` does not implement L1 regularization. Tip: Modify `training_step` to include the L1 norm in the loss function calculation. You don't need to do k-fold cross-validation. Just run your implementation once to check if it works correctly.

```
In [26]: class Mymodel(L.LightningModule):
          def __init__(self, input_shape, num_classes, hidden_units, lambda_L1):
              super().__init__()

              self.hidden_units = hidden_units
```



```

self.lambda_L2 = lambda_L2
self.lambda_L1 = lambda_L1

layers = []
layers.append(nn.Flatten())

for units in hidden_units:
    layers.append(nn.Linear(input_shape, units))
    layers.append(nn.BatchNorm1d(units))
    layers.append(nn.LeakyReLU(negative_slope=0.01))
    input_shape = units

layers.append(nn.Linear(input_shape, num_classes))
self.classifier = nn.Sequential(*layers)

self.loss = nn.CrossEntropyLoss()
self.accuracy = Accuracy(task="multiclass", num_classes=num_classes)

def forward(self, x):
    return self.classifier(x)

def configure_optimizers(self):
    return torch.optim.Adam(
        self.parameters(),
        lr=0.001,
        weight_decay=self.lambda_L2
    )

def training_step(self, batch, batch_idx):
    inputs, labels = batch
    logits = self(inputs)

    ce_loss = self.loss(logits, labels)

    l1_norm = sum(parameter.abs().sum() for parameter in self.parameters())

    loss = ce_loss + self.lambda_L1 * l1_norm

    preds = torch.argmax(logits, dim=1)
    acc = self.accuracy(preds, labels)

    self.log('train_loss', loss, on_step=True, on_epoch=True, logger=True)
    self.log('train_ce_loss', ce_loss, on_step=False, on_epoch=True, logger=True)
    self.log('train_l1_norm', l1_norm, on_step=False, on_epoch=True, logger=True)
    self.log('train_acc', acc, on_step=True, on_epoch=True, logger=True)

    return loss

def validation_step(self, batch, batch_idx):
    inputs, labels = batch
    logits = self(inputs)

    loss = self.loss(logits, labels)

    preds = torch.argmax(logits, dim=1)
    acc = self.accuracy(preds, labels)

```

```

        self.log('val_loss', loss, on_step=True, on_epoch=True, logger=True)
        self.log('val_acc', acc, on_step=True, on_epoch=True, logger=True)

        return loss

    def test_step(self, batch, batch_idx):
        inputs, labels = batch
        logits = self(inputs)

        loss = self.loss(logits, labels)

        preds = torch.argmax(logits, dim=1)
        acc = self.accuracy(preds, labels)

        self.log('test_loss', loss, on_step=True, on_epoch=True, logger=True)
        self.log('test_acc', acc, on_step=True, on_epoch=True, logger=True)

        return loss

    def predict_step(self, batch):
        inputs, labels = batch
        return self(inputs)

hidden_units = [32, 64, 16]
lambda_L2 = 0.0
lambda_L1 = 0.0

model = Mymodel(
    input_shape=dm.input_shape,
    num_classes=dm.num_classes,
    hidden_units=hidden_units,
    lambda_L2=lambda_L2,
    lambda_L1=lambda_L1
)

```

In [27]: `L.seed_everything(42, workers=True)`

```

lambda_L2 = 1e-4
lambda_L1 = 1e-5

l1_model = Mymodel(
    input_shape=dm.input_shape,
    num_classes=dm.num_classes,
    hidden_units=[32, 64, 16],
    lambda_L2=lambda_L2,
    lambda_L1=lambda_L1
)

l1_trainer = L.Trainer(
    max_epochs=3,
    accelerator="gpu" if torch.cuda.is_available() else "cpu",
    devices=1,
    logger=True,
    enable_checkpointing=False
)

```

```

)

l1_trainer.fit(l1_model, dm)

l1_results = l1_trainer.validate(
    l1_model,
    dataloaders=dm.val_dataloader()
)

l1_results

```

Seed set to 42

GPU available: True (mps), used: False

TPU available: False, using: 0 TPU cores

/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/trainer/setup.py:175: GPU available but not used. You can set it by doing `Trainer(accelerator='gpu')`.

💡 Tip: For seamless cloud logging and experiment tracking, try installing [litlogger](https://pypi.org/project/litlogger/) to enable LitLogger, which logs metrics and artifacts automatically to the Lightning Experiments platform.

	Name	Type	Params	Mode	FLOPs
0	classifier	Sequential	28.7 K	train	0
1	loss	CrossEntropyLoss	0	train	0
2	accuracy	MulticlassAccuracy	0	train	0

Trainable params: 28.7 K

Non-trainable params: 0

Total params: 28.7 K

Total estimated model params size (MB): 0.115

Modules in train mode: 14

Modules in eval mode: 0

Total FLOPs: 0

Output()

/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec)`

/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/`val\_dataloader` does not have many workers which may be a bottleneck. Please increase the `num\_workers` argument` to `num_workers=13` in the `DataLoader`

/opt/anaconda3/lib/python3.13/site-packages/lightning/pytorch/`train\_dataloader` does not have many workers which may be a bottleneck. Please increase the `num\_workers` argument` to `num_workers=13` in the `DataLoader`

`Trainer.fit` stopped: `max\_epochs=3` reached.

Output()

Validate metric	DataLoader 0
val_acc_epoch	0.8686666488647461
val_loss_epoch	0.37275394797325134

```
Out[27]: [{'val_loss_epoch': 0.37275394797325134, 'val_acc_epoch': 0.8686666488647461}]
```